SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING
Engineer to Excel

# DATA STRUCTURES

## ASSIGNMENT 1

| | |
|---|---|
| **Name** | D. Hasini |
| **Register No.** | 192425147 |
| **Course Code** | CSA0304 |
| **Course** | Data Structures |
| **Date** | 22 October 2026 |

## A) Define Abstract Data Type (ADT) and explain its features. (5 marks)

### Definition

An Abstract Data Type (ADT) is a mathematical model of a data structure that is defined only by the data it stores and the operations that can be performed on that data. It tells us what the operations do, but it does not tell us how they are implemented. The user of an ADT works only with its interface, while the internal representation (array, linked list, etc.) stays hidden.

For example, a **Stack** ADT is defined by the operations push, pop, peek and isEmpty with the LIFO (Last In First Out) rule. It can be implemented either with an array or with a linked list, and the user of the stack does not need to know which one is used.

### Components of an ADT

- **Data:** the set of values or elements stored in the ADT.
- **Operations:** the set of functions that can be applied on the data, such as insert, delete, search and traverse.
- **Rules (axioms):** the conditions the operations must follow, for example pop on an empty stack is an error.

### Features of an ADT

1. **Abstraction:** Only the essential details (the operations) are shown to the user. The complicated working is hidden.
2. **Encapsulation:** The data and the operations are bundled together as one unit, and the data can be accessed only through the defined operations.
3. **Information hiding:** The internal representation is not visible outside. The implementation can be changed without affecting the programs that use it.
4. **Implementation independence:** The same ADT can be implemented in different ways (array or linked list) and in different languages.
5. **Reusability:** Once an ADT is designed and tested, it can be used in many programs without rewriting it.
6. **Modularity and easy maintenance:** Programs are divided into independent parts, so debugging and modification become easier.
7. **Well-defined interface:** Every operation has a clear name, inputs, outputs and behaviour, which avoids confusion.

### Common examples

List, Stack, Queue, Deque, Priority Queue, Set, Map (Dictionary) and Tree are all ADTs. Each one can be implemented using more than one data structure.

## B) Design an ADT for array and implement in C with complexity analysis. (10 marks)

### Introduction

An array stores elements of the same type in continuous memory locations, and each element is accessed by its index. As an ADT, we describe the array by the data it holds and the operations on it, and we hide the storage details inside a structure.

### ADT specification

**ADT Array**

- **Data:** a fixed-capacity collection of integers data[0..MAX-1] and the count of elements currently stored (size).
- **Operations:** init, insertAt(pos, val), deleteAt(pos), search(key), get(pos), update(pos, val) and display.
- **Rules:** valid positions are 0 to size - 1 (0 to size for insertion). Insertion in a full array and access outside this range are not allowed.

### Operations of the ADT

| Operation | Purpose | Working |
|---|---|---|
| init() | Create an empty array | Sets size = 0 |
| insertAt() | Add an element at a position | Shifts elements right, then stores the value |
| deleteAt() | Remove the element at a position | Shifts elements left to fill the gap |
| search() | Find the index of a key | Compares elements one by one |
| get() / update() | Read or change an element | Direct access using the index |
| display() | Print all elements | Traverses from index 0 to size - 1 |

### Program in C

```
#include <stdio.h>
#define MAX 100

/* ADT: Array */
typedef struct {
    int data[MAX];   /* storage */
    int size;        /* current number of elements */
} Array;

void init(Array *a)
{
    a->size = 0;
}

int insertAt(Array *a, int pos, int val)   /* pos: 0 to size */
{
    int i;
    if (a->size == MAX || pos < 0 || pos > a->size)
        return 0;
    for (i = a->size - 1; i >= pos; i--)
```

```
            a->data[i + 1] = a->data[i];
        a->data[pos] = val;
        a->size++;
        return 1;
    }

    int deleteAt(Array *a, int pos)
    {
        int i;
        if (pos < 0 || pos >= a->size)
            return 0;
        for (i = pos; i < a->size - 1; i++)
            a->data[i] = a->data[i + 1];
        a->size--;
        return 1;
    }

    int search(Array *a, int key)   /* returns index or -1 */
    {
        int i;
        for (i = 0; i < a->size; i++)
            if (a->data[i] == key)
                return i;
        return -1;
    }

    int get(Array *a, int pos)
    {
        return a->data[pos];
    }

    int update(Array *a, int pos, int val)
    {
        if (pos < 0 || pos >= a->size)
            return 0;
        a->data[pos] = val;
        return 1;
    }

    void display(Array *a)
    {
        int i;
        for (i = 0; i < a->size; i++)
            printf("%d ", a->data[i]);
        printf("\n");
    }

    int main()
    {
        Array a;
        int idx;
        init(&a);

        insertAt(&a, 0, 10);
        insertAt(&a, 1, 20);
        insertAt(&a, 2, 30);
        insertAt(&a, 1, 15);
        printf("After insertion : ");
        display(&a);

        deleteAt(&a, 2);
```

```
    printf("After deletion  : ");
    display(&a);

    idx = search(&a, 30);
    if (idx != -1)
        printf("30 found at index %d\n", idx);
    else
        printf("30 not found\n");

    update(&a, 0, 99);
    printf("After update    : ");
    display(&a);
    return 0;
}
```

**Output**

```
After insertion : 10 15 20 30
After deletion  : 10 15 30
30 found at index 2
After update    : 99 15 30
```

**Working**

The array is first filled with 10, 20, 30 and then 15 is inserted at position 1. The elements 20 and 30 are shifted one place to the right, so the array becomes 10 15 20 30. Deleting the element at position 2 (value 20) shifts 30 to the left, giving 10 15 30. Searching for 30 compares 10, 15 and 30, and it is found at index 2. Finally the element at index 0 is updated to 99.

**Complexity analysis**

Let n be the number of elements currently stored in the array.

| Operation | Best case | Average / Worst case | Reason |
|---|---|---|---|
| Access / update by index | O(1) | O(1) | Address is calculated directly |
| Insert at end | O(1) | O(1) | No shifting needed |
| Insert at beginning / middle | O(1) | O(n) | Elements are shifted right |
| Delete at end | O(1) | O(1) | No shifting needed |
| Delete at beginning / middle | O(1) | O(n) | Elements are shifted left |
| Search (linear) | O(1) | O(n) | Key may be at the last place |
| Display / traverse | O(n) | O(n) | Every element is visited once |

**Space complexity:** The array needs O(n) space to store n elements (fixed size MAX in this program). The operations use only a few extra variables such as i, so the extra space is O(1).

**Conclusion:** The array ADT gives very fast access by index, but insertion and deletion in the middle are slow because of shifting. Its size is also fixed, which is its main limitation compared with a linked list.

## C) Explain role of performance analysis in real-time systems. (5 marks)

A real-time system is a system in which the correctness of the result depends not only on the value of the output but also on the time at which it is produced. Examples are an aircraft flight controller, a car airbag system, a pacemaker and an industrial robot. In a hard real-time system a missed deadline can cause a serious failure, while in a soft real-time system (such as video streaming) it only reduces the quality.

Performance analysis means measuring and predicting how fast and how efficiently the system executes its tasks. In real-time systems it is very important because the system must prove that it will always meet its deadlines.

### Role of performance analysis

1. **Meeting deadlines:** It checks that every task completes before its deadline, even in the worst situation.
2. **Worst Case Execution Time (WCET):** Real-time design depends on the worst case time of a task and not on the average time, so analysis finds this upper limit.
3. **Response time and latency:** It measures the delay between an event (sensor input or interrupt) and the output of the system.
4. **Schedulability analysis:** It checks whether a set of tasks can be scheduled together using algorithms like Rate Monotonic (RM) or Earliest Deadline First (EDF) without missing deadlines.
5. **Predictability and reliability:** It makes the system behaviour predictable, which is needed for safety-critical applications.
6. **Efficient use of resources:** It finds the memory use, CPU utilization and power consumption so that limited resources are used properly.
7. **Finding bottlenecks:** It shows which part of the code or hardware is slow, so that it can be optimized by using better algorithms, interrupts and efficient data structures.

### Common performance measures

| Measure | Meaning |
|---|---|
| Execution time | Time taken by a task to complete on the processor |
| Response time | Time from the arrival of an event to its output |
| Throughput | Number of tasks completed in a given time |
| CPU utilization | Percentage of time the processor is busy |
| Jitter | Variation in the timing of a periodic task |

Performance is analysed using timers, profilers, simulators, logic analyzers and mathematical analysis of the algorithms.

**Conclusion:** Performance analysis gives confidence that a real-time system is timely, predictable and safe. Without it, a system may work correctly in testing but fail in actual use because of a missed deadline.